

AI 시스템반도체 시스템 반도체 설계(2기)

프로젝트 결과보고서

가위바위보 미니 프로젝트

정준혁

1. 주제 및 결과 요약
2. 개발 목표 및 개발 결과
3. 핵심 기술
4. 결과 분석 및 기대 효과
5. 향후 연구 과제
6. 프로젝트 수행 후기

주제 및 결과 요약

한 장에서 목적, 입력, 처리 결과를 함께 확인하는 구조

프로젝트 개요

- 웹캠 영상 기반 실시간 2인 가위바위보 시스템 구축
- LiteRT(best.tflite)모델 추론 기반 가위바위보 분류
- Bounding Box 중심 좌표 기반 좌/우 플레이어(P1, P2) 영역 분리 및 규칙 판정

현재 구현 범위

인식 클래스 표시 · 2인 분리 · 3초 카운트다운 · 0.5초 다수결 · 승무패 점수판

처리 흐름

단계	블록 테스트	하단 서브 라벨
1단계	영상	Camera Frame
2단계	추론	LiteRT
3단계	판정	RPS Winner Engine
4단계	점수	UI & Scoreboard

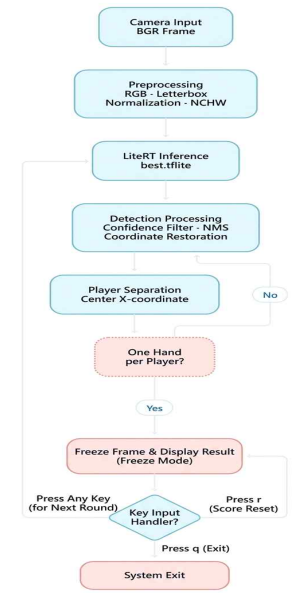
결과 모델 추론 결과를 게임 규칙과 사용자 화면까지 연결한 실행형 미니 프로젝트

개발 목표 및 개발 결과

개발 목표 별 요구 기능과 설계

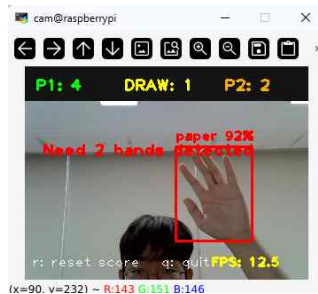
개발 목표	요구 기능	코드 기반 구현
손동작 분류	가위-바위-보 3개 클래스 인식	클래스 ID 0:1:2를 텍스트로 변환
2인 게임	한 프레임에 두 플레이어 검출	박스 중심 x좌표로 좌우 영역 분리
안정적 판정	순간 오인식 영향 완화	종료 전 0.5초 투표의 최빈값 선택
상태 관리	라운드 결과와 점수 제공	승·무·패 누적, 결과 고정, r 키 초기화
2인 플레이 예외 처리	손 감지 예외 시 판정 보류	종료 전 0.5초 투표의 최빈값 선택
동작 고정	라운드 결과 정지 화면 제공	승패 판정 순간 freeze_frame 후 사용자 화면까지 정지

목표 가위, 바위, 보를 안정적으로 인식하고 계산하여 스코어 보드에 출력하는 시스템



개발 목표 및 개발 결과

구현 결과



- **P1 Win/P2 Win** : Bounding Box 중심 좌/우 영역 구분 및 실시간 승패 판정
- **Draw** : 동일 Class 감지 시 무승부 처리로 로직 작동
- **예외판정** : 손 객체가 2개 미만/ 초과 감지 시 게임 보류
- **프리징 및 상태 제어** : 판정 후 화면 정지(Press any key to continue) 및 상단 스코어 보드 연동 (r : 리셋, q : 종료)
- **안정화** : 카운트다운 종료 전 마지막 0.5초 수집 데이터의 최빈값 추출로 순간 오인식 완화

```
# 1단계/2단계: 마지막 0.5초의 최빈값으로 P1/P2 비교
final_p1 = Counter(countdown_state['votes_p1']).most_common(1)[0][0] if countdown_state['votes_p1'] else p1['cid']
final_p2 = Counter(countdown_state['votes_p2']).most_common(1)[0][0] if countdown_state['votes_p2'] else p2['cid']
result = rpsWinner(final_p1, final_p2)
if result == 'P1 Win':
    score_state['p1'] += 1
elif result == 'P2 Win':
    score_state['p2'] += 1
else:
    score_state['draw'] += 1
```

1. YOLOv11 기반 AI 추론 및 후처리

- **입력 규격화** : 320x320 정방형 Letterbox 전처리 적용 -> 손 모양 비율 왜곡 방지
- **후처리 필터링** : CONF_TH(0.4), IOU_TH(0.45) 기준 바운딩 박스 중복 제거 및 좌표 복원



2. 좌/우 플레이어 영역 분리 및 2인 게임 예외 처리

- **좌/우 플레이어 독립 검출** : NMS 박스의 중심 X좌표를 계산하여 중앙 기준으로 P1/P2 분류
- **조건부 예외 처리 로직** : if len(left_boxes) != 1 or len(right_boxes) != 1 구조를 이용하여 영역 내 손 감지 후 판정 유예 및 경고 문구 출력
-> Need 2 hands detected



3. 0.5초 최빈값 다수결 및 상태 관리

1. **현상** : 카운트다운 완료 순간의 1개 프레임 단일 추론 결과만으로 승패를 판정할 때 오인식 발생
사용자가 가위/ 바위/ 보 손 모양을 만드는 과정에서 프레임이 튀어 잘못된 판정 결과가 출력
2. **원인** : 단일 프레임 기분 추론 방식의 한계
카운트다운 종료 직전 순간적인 손 모양 변형, 조면 변화 등에 의한 오분류 위험
3. **문제 해결** : 0.5초 최빈값 다수결 판정 알고리즘 도입
카운트다운 종료 전 마지막 0.5초간 수집된 추론 결과의 평균을 결과값으로 출력
4. **적용 결과** : 순간적인 노이즈에 영향을 받지 않는 안정적인 승패 판정 로직 구축
1회성 순간 오인식을 상쇄하여 실시간 가위바위보 게임 승패 판정 정확도 향상

```
=====
[문제 해결 시각화 : Before vs After 비교 로그]
=====
> 0.5초간 수집된 P1 프레임 기록 : ['rock', 'rock', 'rock', 'rock', 'rock', 'paper']
> 0.5초간 수집된 P2 프레임 기록 : ['paper', 'paper', 'paper', 'paper', 'paper', 'paper']
=====
X [Before] 단일 프레임 (1회성) 판정 : P1(paper) vs P2(paper) -> [Draw]
☑ [After] 0.5초 최빈값 다수결 판정 : P1(rock) vs P2(paper) -> [P2 Win]
=====
```

결과 분석 및 기대효과

■ 모델 성능 평가 및 결과 분석

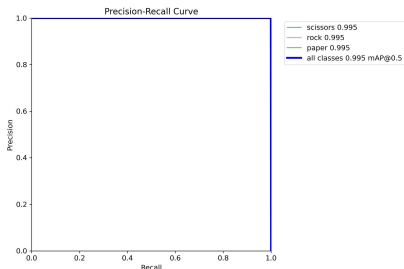
- **mAP@0.5 99.5% 달성** : 전 클래스에서 99.5% 이상의 뛰어난 객체 탐지 성능 기록
- **온디바이스 연산 최적화** : Raspberry Pi 5 환경에서 평균 10~15 FPS 이상 유지하며 안정적인 실시간 게임 진행

■ 기술적 기대 효과

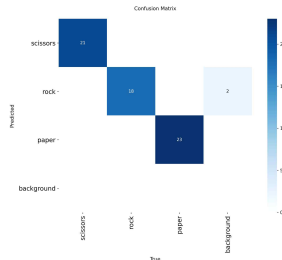
- **임베디드 AI 확장성 검증** : 저전력·제한적 자원 환경에서 LiteRT 기반의 1-Stage Detector 최적화 가능성 입증
- **실시간 프레임 노이즈 상쇄** : 0.5초 최빈값 다수결 및 예외 처리 로직 적용을 통한 오탐 방지

■ 활용 및 사업적 기대 효과

- **Touchless UI/UX** : 비접촉 제어가 필요한 키오스크, 의료기기, 스마트홈 등의 제어 인터페이스로 확장 가능
- **학습 서비스 응용** : 사용자 동작 인식 기반의 교육용 게임 및 인터랙티브 전시·체험 콘텐츠에 전면 적용 가능

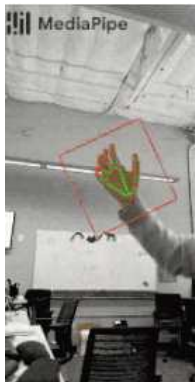


- mAP@0.5 99.5%로 출력
- 단순한 모델 구조로 인해 그래프 선이 사각형에 가까운 이상적인 형태로 출력



- 테스트 이미지 모두 100% 인식
- 배경을 바위로 오탐한 횟수 2회
↳ 안정화 처리를 통해 오탐 완화

- **추후 개선 사항**
 - **배경 오탐 강건성 확보** : 다양한 배경 데이터셋 추가 구축 및 추가적인 튜닝 진행
 - **다양한 환경 변수 대응** : 조명 변화, 극심한 역광, 손 기울어짐 등의 현상에 강건하도록 데이터셋 추가
 - **모델 경량화 및 FPS 향상**
- **후속 개발 주제**
 - **연속 동작 인식** : 단일 컷을 넘어서 연속적인 손동작을 추적하여 계산하는 추적 기술을 개발 -> MediaPipe와 YOLO 연동 등
 - **GPIO 기반 제어** : 라즈베리 파이 GPIO 핀을 연동하여 승 / 패 결과에 따른 LED 점등, 부저 출력 등 직관적인 알림 기능 추가
 - **UI/UX 발전** : 게임 화면 내 승패 이력 그래프 및 애니메이션 효과 추가 가능



출처: chuoing.github.io/mediapipe



프로젝트 수행 후기

- 문제 해결 및 극복 사연
- 단일 프레임 판정의 한계 극복
 - 문제 : 카운트다운 순간 노이즈로 인해 승패가 잘못 판정되는 현상이 발생
 - 해결 : 카운트다운 종료 전 0.5초의 프레임을 수집하고 다수결 알고리즘을 적용하여 오판정률을 낮추었다.
- 손 기울어짐 및 가림 현상으로 인한 오인식 대응 :
 - 문제 : 카메라 각도나 손의 기울어짐 등에 의해 인식률이 떨어지는 현상 발생
 - 임시 해결 : 두 손이 모두 정상 감지 되었을 때만 카운트 다운이 진행되도록 예외처리 로직 추가
 - 추후 해결 : 데이터셋 보충을 통한 강건함 확보
- 배운점 & 느낀점
 - 알고리즘 적 후처리의 중요성 체감 : AI 모델의 정확도도 중요하지만 카메라 입력의 흔들림이나 노이즈를 제어해주는 소프트웨어적 후처리도 신경써야 함을 배웠다.
 - On-Device AI의 흐름 이해 : 라즈베리 파이 환경에서 카메라 영상을 실시간으로 처리하고 UI 및 스코어 보드와 연동하는 과정을 진행하며 전체적인 흐름을 이해할 수 있었다.
- 마무리 소회
 - 수업 시간에 배운 AI 모델 및 파이썬 로직을 바탕으로 발생한 문제를 직접 보정해 보며 수업 내용에 대한 이해와 성취감을 얻을 수 있었다.
 - 향후 MediaPipe 손 관절 추적 기능과 GPIO 하드웨어 제어 등 더 발전된 과정을 경험하며 확장해나가고 싶다.